Built-In Self-Test (BIST) for a 4-bit ALU using Verilog

Soorya Narayan S
Roll Number: B241229EC

July 14, 2026

Abstract

This project implements a Built-In Self-Test (BIST) architecture for a 4-bit Arithmetic Logic Unit (ALU) using Verilog HDL. The design integrates a Linear Feedback Shift Register (LFSR) as a pseudo-random test pattern generator, a finite state machine (FSM) to sequence the ALU through its ADD, SUB, AND, and OR operations, a Multiple Input Signature Register (MISR) to compact the ALU's responses into a single signature, and a comparator that checks the computed signature against a pre-computed golden signature to declare a PASS or FAIL. The complete design was verified through behavioral simulation in Xilinx Vivado (no FPGA implementation or hardware deployment was performed). Fault injection experiments were carried out by deliberately corrupting the ALU logic, and the resulting signatures were used to demonstrate both successful fault detection and a case of signature aliasing, a well-known limitation of signature-analysis-based BIST.

1 Introduction

Built-In Self-Test (BIST) is a design-for-testability (DFT) technique in which a digital circuit is augmented with additional hardware that allows it to test itself, without relying on external Automatic Test Equipment (ATE) to apply test vectors and observe responses. BIST is widely used in VLSI design to reduce test cost, enable at-speed testing, and support field-testing of deployed chips.

A classic BIST scheme, and the one implemented in this project, follows the **Test Pattern Generator (TPG) – Circuit Under Test (CUT) – Output Response Analyzer (ORA)** model:

- An LFSR generates a pseudo-random sequence of test vectors that are applied to the CUT (here, a 4-bit ALU).
- An FSM sequences the CUT through each of its functional operations (ADD, SUB, AND, OR) for a fixed number of test vectors per operation.
- A MISR compresses the long sequence of ALU responses into a fixed-width signature, avoiding the need to store or compare every individual response.
- A comparator checks the final signature against a golden signature (the expected signature for a fault-free circuit) and asserts a PASS/FAIL flag.

This project was undertaken independently, outside of any coursework, to gain hands-on experience with DFT concepts and to build a project suitable for showcasing practical Verilog and VLSI testing skills.

2 Tools and Simulation Environment Used

- **HDL:** Verilog
- **Design Suite:** Xilinx Vivado
- **Simulator:** Vivado Simulator (XSim), behavioral simulation
- **Scope:** RTL design and functional (behavioral) simulation only – no synthesis, implementation, or hardware deployment on an FPGA board was carried out. All results in this report are derived purely from simulation waveforms.

3 Theory of Operation

3.1 Linear Feedback Shift Register (LFSR)

The LFSR is an 8-bit shift register with linear feedback taps that generates a maximal-length pseudo-random bit sequence. On every active clock edge (while enabled), the register shifts right and a new feedback bit – computed as the XOR of specific tap positions – is inserted at the most significant bit. The 8-bit LFSR output is split into two 4-bit operands, $A[3:0]$ and $B[3:0]$, which are fed to the ALU as test vectors. Because the same feedback polynomial and seed are used every run, the LFSR produces a deterministic, repeatable pseudo-random test sequence, which is essential for a golden signature to be meaningful.

3.2 BIST Controller (Finite State Machine)

The controller is a Moore-type FSM with five states: `ADD_TEST`, `SUB_TEST`, `AND_TEST`, `OR_TEST`, and `DONE`. In each test state, the FSM drives a fixed 2-bit opcode to the ALU and applies 255 test vectors (counted using an internal 8-bit counter) before advancing to the next operation. Once all four operations have been exercised, the FSM enters the `DONE` state and remains there, halting further testing.

3.3 4-bit ALU (Circuit Under Test)

The ALU is a simple combinational unit supporting four operations selected by a 2-bit opcode: addition, subtraction, bitwise AND, and bitwise OR. It produces a 4-bit result and a carry/borrow flag, giving a combined 5-bit response for each test vector.

3.4 Multiple Input Signature Register (MISR)

The MISR is a 5-bit register that compacts the sequence of ALU responses into a single signature over the course of the test. On every active clock edge (while enabled), each bit of the MISR is updated as the XOR of a neighboring bit and the corresponding ALU response bit, effectively folding the entire 1020-vector response sequence (255×4 operations) into a 5-bit signature. This is a form of response compaction that trades a small, controlled probability of *aliasing* (described in Section 8) for a drastic reduction in the amount of data that must be stored or compared.

3.5 Comparator and Golden Signature

The golden signature is the signature value obtained by simulating a fault-free ALU over the full test sequence. For this design, the golden signature was determined to be `5'b01100`. The comparator continuously compares the live MISR signature against this golden value and asserts `pass` when they match.

4 System Architecture

Figure 1 shows the complete BIST architecture, illustrating the flow of data from the LFSR-generated test vectors, through the ALU under the control of the BIST FSM, into the MISR, and finally to the comparator that produces the PASS/FAIL result.

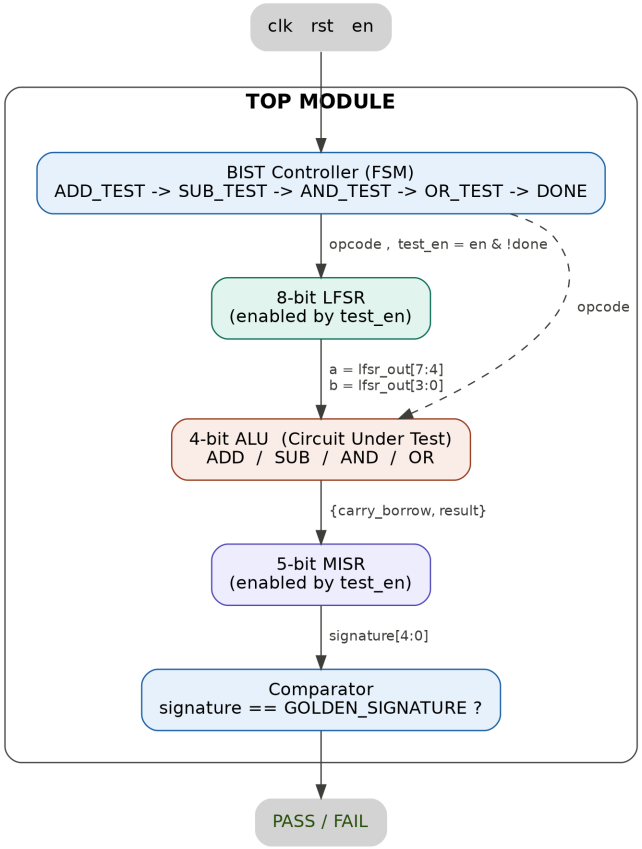


Figure 1: Final BIST architecture – signal flow between the BIST controller, LFSR, ALU, MISR, and comparator.

5 Data Flow

While Figure 1 shows the static structure of the design, it is also useful to trace the dynamic sequence of events that occurs during a single BIST run, from reset to the final PASS/FAIL decision. Figure 2 presents this as a data-flow diagram: after reset, the FSM begins in the `ADD_TEST` state, and a core loop – LFSR shift → operand split → ALU operation → MISR update – repeats once per clock cycle for all 255 vectors across each of the four operations (1020 cycles in total). Once this loop completes, the FSM transitions to `DONE`, which deasserts `test_en` and freezes both the LFSR and the MISR. The frozen signature is then compared against the golden signature to produce the final `PASS/FAIL` result.

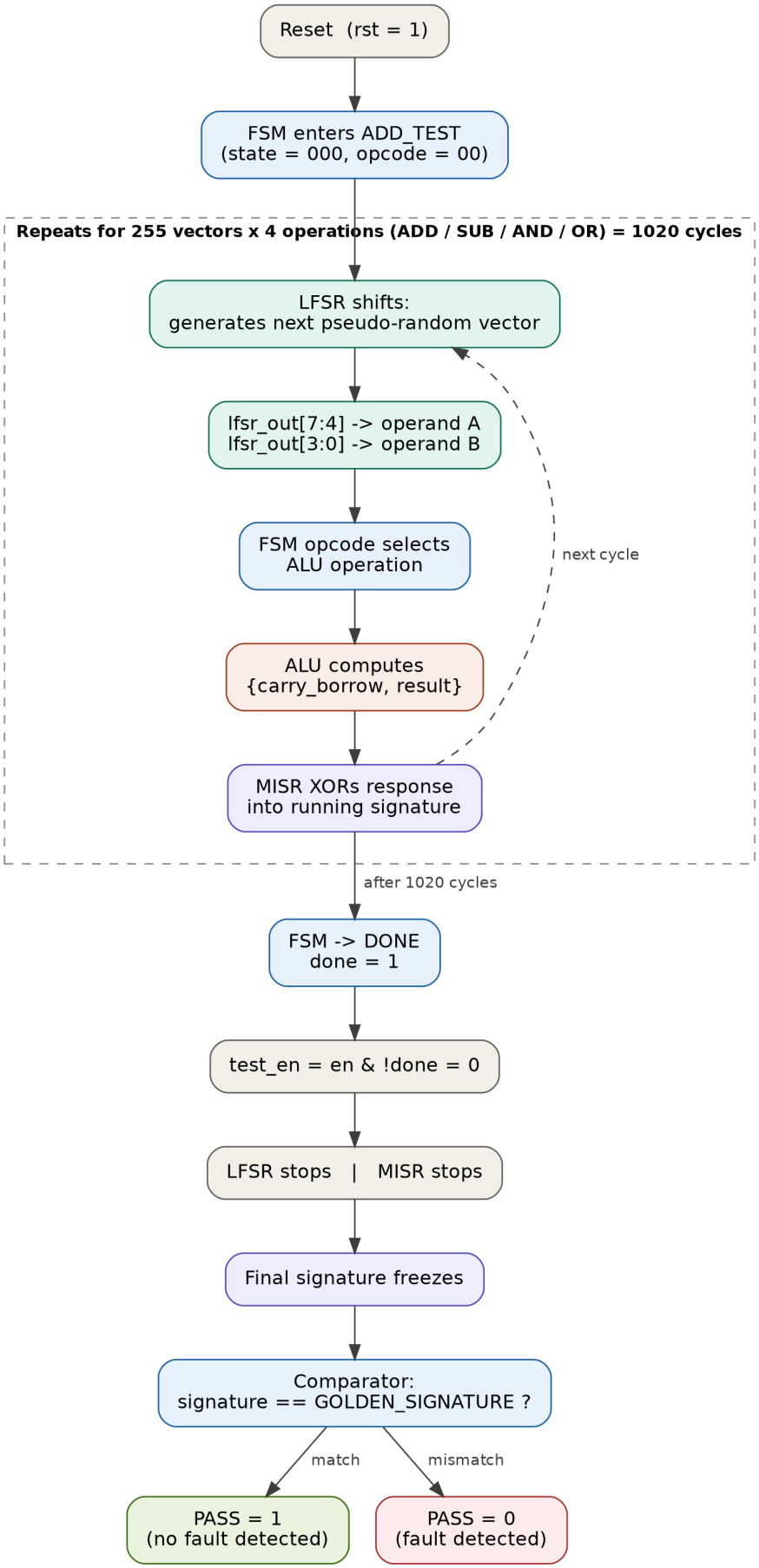


Figure 2: Data-flow diagram of a complete BIST run, from reset through the 1020-cycle test loop to the final PASS/FAIL comparison.

6 Verilog Source Code

The complete RTL for the design, consisting of six modules – `top`, `FSM`, `lfsr`, `alu`, `misr`, and `comparator` – is listed below.

6.1 Top-Level Module

```
module top(
    input clk,
    input en,
    input rst,
    output pass,
    output [4:0] signature
    );
    parameter [4:0] golden_signature = 5'b01100;
    wire [7:0]lfsr_out;
    wire [1:0] opcode;
    wire [4:0]alu_out;
    wire [3:0]result;
    wire carry_borrow;
    wire done;
    wire misr_en;
    assign alu_out={carry_borrow,result};
    assign test_en = en && !done;
    //lfsr
    lfsr l1(
    .clk(clk),
    .rst(rst),
    .en(test_en),
    .lfsr_out(lfsr_out)
);
    //fsm
    FSM f1(
    .clk(clk),
    .rst(rst),
    .en(en),
    .opcode(opcode),
    .done(done)
);
    //alu
    alu a1(
    .a(lfsr_out[7:4]),
    .b(lfsr_out[3:0]),
    .opcode(opcode),
    .result(result),
    .carry_borrow(carry_borrow)
);

    //misr
    misr m1(
    .clk(clk),
    .rst(rst),
    .en(test_en),
    .alu_out(alu_out),
    .signature(signature)
);
    //comparator
    comparator c1(
    .signature(signature),
    .golden_signature(golden_signature),
    .pass(pass)
);
```

```
endmodule
```

Listing 1: Top-level BIST integration module (top.v)

6.2 BIST Controller (FSM)

```
module FSM(
    input clk,
    input rst,
    input en,
    output reg [1:0]opcode,
    output done
    );
 parameter NUM_VECTORS = 8'd255;
 parameter ADD_TEST=3'b000;
 parameter SUB_TEST=3'b001;
 parameter AND_TEST=3'b010;
 parameter OR_TEST=3'B011;
 parameter DONE=3'b100;
 reg [2:0] state;
 reg [7:0] count;
 always@(posedge clk or posedge rst)begin
 if (rst)begin
 state<=ADD_TEST;
 count<=8'd0;
 opcode<=2'b00;
 end
 else if(en)begin
 case(state)
 ADD_TEST:begin
 opcode<=2'b00;
 if (count==NUM_VECTORS-1)begin
 count<=8'b0;
 state<=SUB_TEST;
 end
 else  count<=count+1;
 end
  SUB_TEST:begin
 opcode<=2'b01;
 if (count==NUM_VECTORS-1)begin
 count<=8'b0;
 state<=AND_TEST;
 end
 else  count<=count+1;
 end
  AND_TEST:begin
 opcode<=2'b10;
 if (count==NUM_VECTORS-1)begin
 count<=8'b0;
 state<=OR_TEST;
 end
 else  count<=count+1;
 end
  OR_TEST: begin
 opcode<=2'b11;
 if (count==NUM_VECTORS-1)begin
 count<=8'b0;
 state<=DONE;
 end
 else  count<=count+1;
 end
 DONE:state<=DONE;
```

```
 default:
begin
    state <= ADD_TEST;
    count <= 8'd0;
    opcode <= 2'b00;
end
 endcase
 end
 end
 assign done = (state == DONE);
endmodule
```

Listing 2: BIST controller finite state machine (FSM.v)

6.3 Pseudo-Random Test Pattern Generator (LFSR)

```
module lfsr(
    input clk,
    input en,
    input rst,
    output reg [7:0] lfsr_out
    );
    wire feedback;
    assign feedback=lfsr_out[7]^lfsr_out[5]^lfsr_out[4]^lfsr_out[3];
    always@(posedge clk or posedge rst)
    begin
    if(rst)
    lfsr_out<=8'b10101100 ;
    else if(en)
    lfsr_out<={feedback,lfsr_out[7:1]};
    end

endmodule
```

Listing 3: 8-bit Linear Feedback Shift Register (lfsr.v)

6.4 Circuit Under Test (ALU)

```
module alu(
    input [3:0] a,
    input [3:0] b,
    input [1:0] opcode,
    output reg [3:0] result,
    output reg carry_borrow
    );
always @(*) begin
case(opcode)
    2'b00:  {carry_borrow,result}=a+b;
    2'b01: {carry_borrow,result}=a-b;
    2'b10: begin result=a^b;carry_borrow=0;end
    2'b11: begin result=a|b;carry_borrow=0;end
    default: begin result = 4'b0000;carry_borrow = 0; end
endcase
end
endmodule
```

Listing 4: 4-bit ALU under test (alu.v)

6.5 Response Compactor (MISR)

```
module misr(
    input clk,
    input en,
    input rst,
    input [4:0] alu_out,
    output reg [4:0] signature
    );
    always@(posedge clk or posedge rst)begin
    if(rst)
    signature<=5'b00000;
    else if(en) begin
    signature[4]<=signature[0]^alu_out[4];
    signature[3]<=signature[4]^^alu_out[3];
    signature[2] <= signature[3] ^ alu_out[2];
    signature[1] <= signature[2] ^ alu_out[1];
    signature[0] <= signature[1] ^ alu_out[0];
    end
end
endmodule
```

Listing 5: 5-bit Multiple Input Signature Register (misr.v)

6.6 Signature Comparator

```
module comparator(
    input [4:0] signature,
    input [4:0] golden_signature,
    output pass
    );
    assign pass=signature==golden_signature;
endmodule
```

Listing 6: Golden signature comparator (comparator.v)

7 Module Descriptions

7.1 Top-Level Module

Module Name: `top`
Description: Integrates all sub-modules of the BIST architecture. It instantiates the LFSR, the FSM controller, the ALU, the MISR, and the comparator, wiring the LFSR output to the ALU operands, the FSM opcode to the ALU's operation select line, the ALU's combined result and carry/borrow to the MISR input, and the MISR signature to the comparator. The module exposes the final `pass` flag and the live `signature` value as outputs.

7.2 BIST Controller (FSM)

Module Name: `FSM`
Inputs: Clock (`clk`), Reset (`rst`), Enable (`en`)
Outputs: Opcode (`opcode`), Done flag (`done`)
Description: Sequences the ALU through its four operations – ADD, SUB, AND, and OR – applying 255 pseudo-random test vectors to each operation before moving to the next. An internal 8-bit counter tracks the number of vectors applied within each state, and a 3-bit state register tracks which operation is currently under test. Once all four operations have been

tested, the FSM transitions to a `DONE` state and halts, which in turn gates off the LFSR and MISR clock-enables via the top-level `test_en` signal.

7.3 Pseudo-Random Test Pattern Generator (LFSR)

Module Name: `lfsr`
Inputs: Clock (`clk`), Enable (`en`), Reset (`rst`)
Output: 8-bit pseudo-random pattern (`lfsr_out`)
Description: Implements an 8-bit Fibonacci LFSR with feedback taps at bit positions 7, 5, 4, and 3. On reset, the register is loaded with a fixed non-zero seed (`8'b10101100`) to guarantee a deterministic, repeatable pseudo-random sequence. The upper and lower nibbles of the output are used as the two ALU operands for each test vector.

7.4 Circuit Under Test (ALU)

Module Name: `alu`
Inputs: Operand A (`a`), Operand B (`b`), Opcode (`opcode`)
Outputs: Result (`result`), Carry/Borrow (`carry_borrow`)
Description: A purely combinational 4-bit ALU that performs addition, subtraction, bitwise AND, or bitwise OR depending on the 2-bit opcode. Addition and subtraction produce a carry-out/borrow-out bit in addition to the 4-bit result, while the logical operations tie the carry/borrow output to zero. This is the module whose faulty behavior is exercised in the fault-injection experiments of Section 8.

7.5 Response Compactor (MISR)

Module Name: `misr`
Inputs: Clock (`clk`), Enable (`en`), Reset (`rst`), ALU response (`alu_out`)
Output: Signature (`signature`)
Description: Compacts the 5-bit ALU response (result concatenated with carry/borrow) generated on every clock cycle into a single 5-bit running signature, using a shift-and-XOR feedback structure. Over the full test sequence of 1020 vectors (255 vectors $\times$ 4 operations), this reduces the entire response stream to one 5-bit value that can be checked with a single comparison, at the cost of a small probability of aliasing.

7.6 Signature Comparator

Module Name: `comparator`
Inputs: Live signature (`signature`), Golden signature (`golden_signature`)
Output: Pass flag (`pass`)
Description: A simple combinational equality comparator that asserts `pass` whenever the live MISR signature matches the pre-computed golden signature for a fault-free ALU.

8 Simulation Results

All results below were obtained from behavioral simulation in Vivado XSim, using dedicated testbenches for each module (`lfsr_tb`, `alu_tb`, `misr_tb`, `FSM_tb`) as well as a full-system testbench (`top_tb`) that applies a reset, enables the BIST controller, and runs the complete 1020-vector test sequence.

8.1 Fault-Free Operation

With the ALU operating correctly, the BIST controller cycles through all four operations, and the final MISR signature matches the golden signature (`5'b01100`), asserting `pass = 1` once the FSM reaches its `DONE` state, as shown in Figure 3.

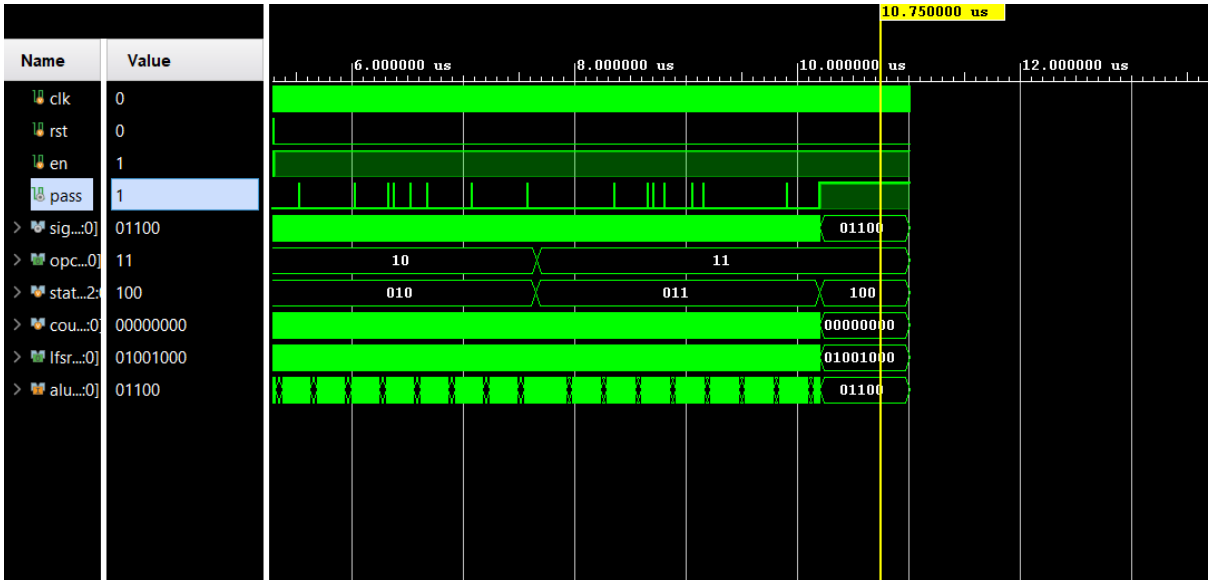


Figure 3: Simulation waveform for a fault-free ALU: the FSM cycles through states `010` through `100`, the final signature settles at `01100`, and `pass` is asserted.

8.2 Fault Detection

To validate the BIST's fault-detection capability, a fault was deliberately injected into the ALU by replacing the bitwise AND operation with an addition (`+`) operation. This produces a different sequence of ALU responses during the AND-test phase, which propagates through the MISR to produce a signature (`5'b10101`) that does *not* match the golden signature. As shown in Figure 4, `pass` correctly drops to `0`, demonstrating that the BIST successfully detects this fault.

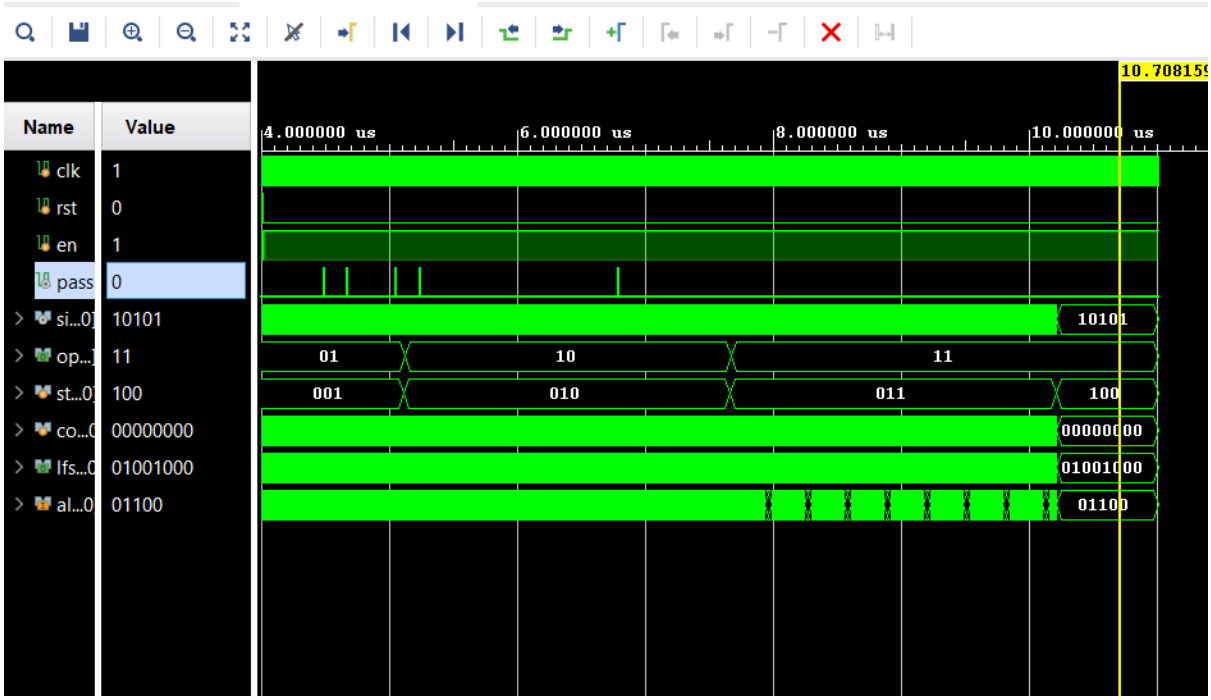


Figure 4: Simulation waveform with a `+` instead of `&` fault injected into the ALU: the resulting signature (`10101`) mismatches the golden signature, and `pass` correctly deasserts.

8.3 Signature Aliasing

A second fault was injected by replacing the bitwise AND operation with a bitwise XOR operation. Despite the ALU producing incorrect responses during the AND-test phase, the final MISR signature happened to settle back to the same value as the golden signature (`5'b01100`), causing `pass` to incorrectly assert, as shown in Figure 5. This is an example of **signature aliasing**, an inherent limitation of any response-compaction scheme: because a MISR maps a very long response sequence onto a small signature space, it is mathematically possible –though generally of low probability – for two different (one faulty, one fault-free) response sequences to compact to the identical signature. This result was intentionally reproduced to demonstrate an understanding of the trade-offs involved in signature-analysis-based BIST, rather than as an unexpected failure of the design.

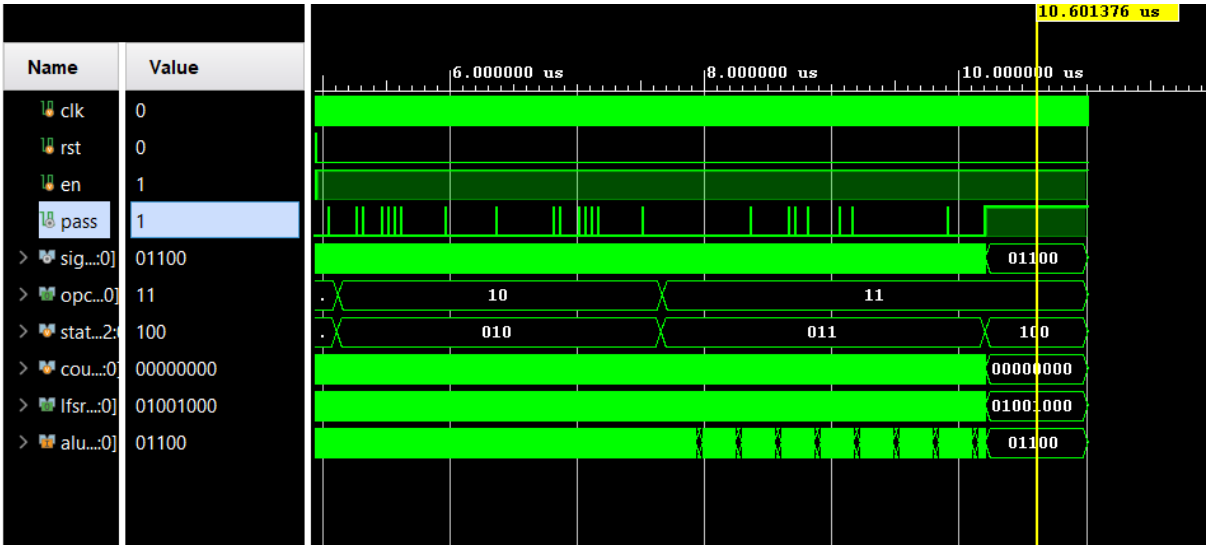


Figure 5: Simulation waveform with an `xor` instead of `&` fault injected into the ALU: the faulty response sequence aliases to the same signature as the golden signature, causing `pass` to incorrectly assert.

9 Conclusion

This project successfully demonstrates a complete Built-In Self-Test architecture for a 4-bit ALU, integrating a pseudo-random test pattern generator (LFSR), a test sequencing controller (FSM), the circuit under test (ALU), a response compactor (MISR), and a golden-signature comparator, entirely verified through Verilog behavioral simulation. Fault-injection experiments confirmed that the design correctly flags a faulty ALU in the general case, while also surfacing a real instance of signature aliasing – illustrating both the effectiveness and the inherent probabilistic limitation of signature-analysis-based BIST. As the project was carried out at the simulation level only, natural extensions would include synthesizing the design for an FPGA target, characterizing the aliasing probability analytically, and comparing MISR-based compaction against alternative response analysis schemes.